1. Identity Cluster — "Who is this user?"

AppUser is the root of everyone's identity — every teacher, student, and admin has exactly one row here. It only holds things every account needs regardless of role: email, password, role flag, active status.

Teacher, Student, PlatformAdmin are each a 1:1 extension of AppUser. Think of it like this: AppUser answers "can this person log in and what role are they?" while the extension table answers "given that they're a Teacher, what else do we know about them?" (full name, phone). We split it this way instead of one giant table because a Student needs a `ParentGuardianEmail` that means nothing for a Teacher — mixing them would leave half the columns null depending on role, which is messy and error-prone.

Relationship: `Teacher.UserId`, `Student.UserId`, `PlatformAdmin.UserId` are all *both* the primary key of their own table *and* a foreign key back to `AppUser.Id`. That's what makes it a true 1:1 — a Teacher row can't exist without a matching AppUser row.

2. Subscription Cluster — "What can this teacher do, and how much have they used?"

SubscriptionPlan is just a catalog: "Free plan allows 30 students, 5 exams/month." The Super Admin edits this table; nothing else does.

TeacherSubscription is the join between a specific Teacher and a specific Plan, with a start/end date. Why not just put `PlanId` directly on the Teacher table? Because plans change over time — a teacher might upgrade in March. Keeping it as its own table gives you a *history*, not just a current snapshot. The "current plan" is simply the row with `Status = Active`.

UsageCounter answers "has this teacher hit their limit *this month*?" without having to count up every exam and file they've ever created every time they try to generate a new exam. It's a running tally, refreshed monthly, purely for performance.

Relationship: Teacher (1) → many TeacherSubscriptions over its lifetime → each pointing to one SubscriptionPlan. Teacher (1) → many UsageCounters, one per calendar month.

3. Classroom Cluster — "Who's in whose class?"

Subject is a simple shared list — "Math," "Arabic," "Physics." It exists purely so every teacher isn't typing free text that can't be grouped or reported on later.

Classroom is where a Teacher and a Subject actually meet — "Mr. Ahmed's Grade 10 Physics class." One teacher owns it, one subject it's about.

Enrollment is the *bridge* between Students and Classrooms. This is the classic many-to-many situation: one student can be in five classrooms (different teachers, different subjects), and one classroom holds many students. You can't represent that with a simple foreign key on either side — you need a junction table sitting in the middle, and that junction table is also a natural place to record *when* they joined.

Relationship: Teacher (1) → many Classrooms. Subject (1) → many Classrooms. Classroom ←→ Student is many-to-many, resolved through Enrollment.

4. Content Cluster — "What are they learning from?"

LearningMaterial is the *concept* of a lesson — "Chapter 3: Newton's Laws" — owned by a classroom.

MaterialVersion is every time that lesson gets uploaded or re-uploaded. This is the important one: we don't overwrite the file when a teacher updates it — we add a new version and mark it current. Why? Because an Exam generated last week points to *last week's version*. If the teacher fixes a typo today, we don't want that exam's questions to suddenly reference content that's changed underneath it.

MaterialChunk exists because the actual AI search doesn't happen in SQL Server — it happens in Qdrant (a vector database). But we still want a lightweight record in our own database saying "this chunk of text, from this version, corresponds to this vector ID over in Qdrant." It's a bridge, not a duplicate.

VideoDetail is a 1:1 add-on to LearningMaterial that only exists when the material type is Video — extra fields like duration and stream URL that a PDF simply doesn't need.

Relationship: Classroom (1) → many LearningMaterials → each has many MaterialVersions (its edit history) → each version has many MaterialChunks (its RAG pieces). LearningMaterial (1) → optionally one VideoDetail.

5. Exam & Question Cluster — this is the heart of the system

Question is a single reusable question, owned by a Teacher — it lives in their personal "question bank," independent of any one exam. This is deliberate: a teacher might reuse the same question across three different exams.

QuestionOption holds the answer choices for MCQ/True-False questions — one row per option, with a flag for which one is correct.

QuestionRubric is a 1:1 add-on to Question, only for Essay/Short-Answer types — it holds the AI-generated grading criteria plus whatever extra instructions the teacher layered on top.

Exam is the container a teacher publishes to students — it belongs to one classroom and optionally points back to the MaterialVersion it was generated from.

ExamQuestion is the bridge between Exam and Question — and this is the second many-to-many in the system. The same Question can appear on multiple Exams, and an Exam obviously has multiple Questions. This junction table also carries exam-specific detail like point value and order — because a question might be worth 5 points on one exam and 10 on another.

Relationship: Teacher (1) → many Questions. Question (1) → many QuestionOptions, and optionally one QuestionRubric. Classroom (1) → many Exams. Exam ←→ Question is many-to-many, resolved through ExamQuestion.

6. Attempt & Grading Cluster — "What did the student actually do, and who's responsible for the score?"

ExamAttempt is one student sitting down and taking one exam — it tracks timing, status, and total score.

StudentAnswer is one row per question *within* that attempt — this is where the student's actual answer and the AI's score live.

GradeOverride is not a status flag — it's a permanent, append-only log entry. Every time a teacher changes an AI score, instead of just updating a number and losing the original, we write a new row here: old score, new score, who did it, when, why. That's what "auditable" actually means in a database — you can't overwrite history, you record on top of it.

Relationship: Exam (1) → many ExamAttempts (one per student). ExamAttempt (1) → many StudentAnswers (one per question). StudentAnswer (1) → potentially many GradeOverrides (usually zero or one, but the table allows more in case a teacher changes their mind twice).

7. Anti-Cheating, Notifications, Chat, Parent Reports, Audit — the "supporting cast"

- **AntiCheatingEvent** hangs off ExamAttempt — every tab-switch or app-minimize during that specific attempt becomes a row, and the sequence number is what tells the app "this is the 1 st, so warn" vs "this is the 2 nd, so flag."
- **Notification** hangs off AppUser (any role) — general in-app/push/email alerts.
- **ParentReportLog** hangs off both Student and ExamAttempt (one row per attempt) — it's kept separate from Notification because it has different rules (it always goes to the guardian email, not a logged-in user).
- **ChatMessage** hangs off Classroom (which room the message is in) and AppUser (who sent it).
- **AuditLog** is the platform's general-purpose "who changed what" table for anything *not* already covered by GradeOverride — plan changes, deletions, admin actions.

Relationship (in one sentence each): ExamAttempt (1) → many AntiCheatingEvents. AppUser (1) → many Notifications. Student (1) → many ParentReportLogs, and each one points to exactly one ExamAttempt. Classroom (1) → many ChatMessages, each sent by one AppUser. AppUser (1) → many AuditLog entries as the "performed by."

8.Payment Cluster — "Who paid for what, and how do we know it's real?"

ClassroomPricing is a simple 1:1 add-on to Classroom — just "how much does this cost to join." We didn't just add a `Price` column directly onto the `Classroom` table because pricing is a *different concern* from what a classroom *is* (its name, subject, teacher). Keeping it separate means a teacher can change the price next month without that change reaching backward and rewriting what a student paid last month — the actual payment amount lives somewhere else (see below), completely unaffected by future re-pricing.

PaymentTransaction is the record of one specific payment attempt — a student trying to pay for one specific classroom. This is intentionally *not* the same thing as being enrolled. Think about why: a student can click "pay," get redirected to Paymob, and then abandon the page, get their card declined, or close the app. None of that should ever make them a member of the classroom. So PaymentTransaction sits in a `Pending` state and only becomes `Paid` once we get independent proof — which brings us to the next table.

PaymentWebhookLog exists because of a subtle but important trust problem: your own app's frontend is not a reliable witness to whether money actually moved. A student's browser could say "payment succeeded" even if it didn't — maliciously or just due to a network hiccup. The *only* thing we trust is a signed message sent directly from Paymob's servers to ours (a webhook). This table logs every single one of those messages — even the ones we reject or ignore — because if a payment is ever disputed, you need to be able to say "here's exactly what Paymob told us, and when." It's not really a business entity like the others; it's closer to a black-box flight recorder for the payment flow.

TeacherPayoutStatement exists because Draya — not the individual teacher — is the one actually collecting the money (that's what "merchant of record" means in the assumption I flagged earlier). So there needs to be a table that periodically says "here's what teacher X earned this month, here's Draya's cut, here's what we owe them." It's the payout side of the ledger, separate from the collection side.

How it connects to what you already know:

- **Classroom (1) → (1) ClassroomPricing** — every classroom has exactly one current price (which can be zero, i.e., free).
- **Student (1) → (many) PaymentTransaction, Classroom (1) → (many) PaymentTransaction** — a student can attempt payment for many classrooms over time, and a classroom can be paid for by many different students. Each attempt is its own row.

- **PaymentTransaction (1) → (0 or 1) Enrollment** — this is the important link back to the cluster you already understand. The existing `Enrollment` table (Student ↔ Classroom) now has an optional pointer back to the transaction that paid for it. For a free classroom, that pointer is just empty — nothing else changes. For a paid classroom, the `Enrollment` row *cannot exist* until that pointer has something valid in it, and the only process allowed to fill it in is the webhook handler after it verifies payment. That's the whole security model in one sentence: **no verified webhook, no Enrollment row, no matter what the app's UI is showing the student.**
- **Teacher (1) → (many) TeacherPayoutStatement** — one statement per teacher per billing period, built by aggregating all their `Paid` PaymentTransactions in that window.